About this PDF

Built only from: **NOC:Computer Networks and Internet Protocol (NPTEL course book, weeks 3, 5, 7)** — https://nptel.ac.in/courses/106105183

Licence: CC BY-SA 4.0. Accessed 2026-09-26.

Covers 6 of 7 syllabus sub-topics of this unit.

Partly covered in the source:

- Congestion avoidance (DECbit, RED)

Not in the source (not covered in this PDF):

- SCTP

Transport-Layer Protocols: UDP

Definition

UDP is a transport-layer protocol used by certain Internet applications; unlike TCP, it does not provide reliability, flow control, or congestion control mechanisms.

Explanation

UDP offers a lightweight service where data is sent without the overhead of establishing a connection or managing acknowledgments. Because it lacks the reliability and control features of TCP, it is suitable for applications that can tolerate loss or that implement their own handling of such issues.

Key points

- UDP is employed by specific applications that do not need the reliability guarantees of TCP.
- It does **not** support reliability, flow control, or congestion control.

TCP: Connection Management

Definition

TCP manages a connection using several timers that control retransmission, probe the receiver, and close idle connections.

Explanation

When a segment is sent, TCP starts a retransmission timer. If the acknowledgement is not received before the timer expires, the segment is retransmitted and the timeout value (RTO) is doubled for the next attempt. This exponential back-off continues (RTO, 2 × RTO, 4 × RTO, …) until an acknowledgement arrives, after which the RTO is reset to its original value. Additional timers support other aspects of connection management: the persistent timer probes a zero-window receiver, the keepalive timer terminates a connection that has been idle for a long period, and the time-wait state holds the connection for roughly twice the maximum packet lifetime before final closure.

Example

If a connection remains idle after being established, the keepalive timer will eventually fire and cause the connection to be closed automatically.

Algorithm / Steps

1. Send a segment and start the retransmission timer.
2. If the timer expires without an acknowledgement, retransmit the segment and set the next timeout to twice the previous RTO.
3. Repeat step 2, doubling the timeout each time (4 × RTO, 8 × RTO, …) until an acknowledgement is received.
4. Upon receiving the acknowledgement, reset the RTO to its original value.
5. If the receiver’s advertised window is zero, the persistent timer expires and a probe packet is sent to obtain an updated window size.
6. If the connection remains idle for a long duration, the keepalive timer expires and the connection is closed.
7. After a normal closure, the connection enters the time-wait state for approximately twice the packet lifetime before being fully released.

Key points

- Retransmission timeout uses exponential back-off (RTO, 2 × RTO, 4 × RTO, …).
- Persistent timer probes a zero-window receiver to avoid deadlock.
- Keepalive timer closes connections that stay idle too long.
- Time-wait state delays final closure for about two packet lifetimes.

Flow control

Definition

Flow control is a mechanism used by transport layer protocols to regulate the amount of data the sender can transmit so that the receiver’s buffer does not overflow and the network is not overloaded.

Explanation

The sender maintains a window size that limits the number of unacknowledged packets it may have in the network. This window is adjusted based on feedback from the receiver, called the receiver advertised window, which indicates the available buffer space at the receiver. The sender’s transmission rate is therefore the minimum of the rate indicated by the receiver window and the rate supported by the network, which is reflected in the congestion window. By increasing the window after receiving acknowledgments and decreasing it when a timeout occurs, the protocol can pipeline multiple packets while preventing loss due to buffer overflow.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

Example

In a sliding window protocol, the sender initially transmits one packet. After the acknowledgment for that packet is received, the sender increases its window and sends three packets in parallel; later the window is increased again to send four packets, allowing multiple packets to be in flight without exceeding the link capacity.

Algorithm / Steps

1. Sender transmits a packet while the number of outstanding packets is less than the current window size.
2. Receiver acknowledges the packet and advertises its available buffer space (receiver window).
3. Sender updates its window size to the minimum of the receiver window and the congestion window.
4. If a timeout occurs, the sender retransmits the timed-out packet and may reduce the window size.
5. The process repeats, allowing pipelined transmission of multiple packets.

Key points

- Flow control prevents receiver buffer overflow by using a receiver advertised window.
- Sliding window protocols allow multiple packets to be sent in parallel, improving link utilization.
- The sender’s window is the minimum of the receiver window and the congestion window.
- Timeouts trigger retransmission and possible reduction of the sending window.

Congestion Control

Definition

Congestion control is the mechanism by which TCP adjusts its sender window size to match the capacity of the network, initially starting with a minimal congestion window and then increasing it gradually until a bottleneck is discovered; when congestion is detected, the rate is decreased using the additive increase-multiplicative decrease (AIMD) principle.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

Explanation

TCP treats packet loss as a signal of congestion. When loss is observed, TCP reduces its sending rate rather than trying to prevent congestion entirely; it first detects congestion by monitoring losses and then updates its rate to alleviate the condition. The AIMD algorithm provides both efficiency and fairness: the congestion window is increased additively while bandwidth is available, and upon detecting loss the window is reduced multiplicatively, which helps achieve max-min fairness among competing flows. The sending rate can be approximated as the congestion window divided by the round-trip time (RTT), linking window size directly to the transmission rate.

Example

When a receiver obtains out-of-order segments (e.g., packets 1, 2, 3 received, packet 4 lost, then packets 5, 6, 7 arrive), it sends duplicate acknowledgments for the last in-order packet (ACK 3). Receiving three duplicate ACKs leads TCP to infer that packet 4 is lost, trigger fast retransmission, and invoke the congestion control response.

Key points

- TCP’s congestion control combines flow control and congestion control by managing the sender’s congestion window.
- Packet loss is interpreted as congestion, prompting a reduction of the sending rate rather than trying to avoid congestion altogether.
- The AIMD principle (additive increase, multiplicative decrease) underlies TCP’s approach, balancing link utilization and fairness.
- Duplicate acknowledgments provide an early indication of loss, enabling faster congestion response than waiting for a retransmission timeout.

Congestion avoidance (DECbit, RED)

Partly covered in the source.

Definition

Congestion avoidance is the phase in TCP where, after the slow start threshold is crossed, the sender increases its congestion window using an additive increase method rather than exponential growth.

Explanation

When the congestion window reaches the slow start threshold, TCP switches from exponential growth to a linear increase. This additive increase is usually implemented by increasing the congestion window by a fraction of the maximum segment size for each acknowledged segment, approximated by the formula: new window = current window + (MSS × MSS) / current window. The purpose is to probe for available bandwidth while limiting the risk of causing congestion.

Algorithm / Steps

1. Start with slow start and increase the congestion window exponentially until the slow start threshold is reached.

2. After crossing the threshold, apply additive increase: for each ACK, increase the congestion window by (MSS × MSS) / current window.
3. If packet loss is detected (e.g., by triple duplicate ACKs), set the slow start threshold to half of the current congestion window and reduce the congestion window to this new threshold, then continue with additive increase.

Key points

- Congestion avoidance follows slow start and uses linear growth of the congestion window.
- The increase is based on a formula that adds a fraction of the maximum segment size for each ACK.
- Detection of loss causes the slow start threshold to be halved and the window to be reduced, after which additive increase resumes.

Quality of Service

Definition

Quality of Service (QoS) refers to the capability of a network to provide better service to selected network traffic, such as voice or video, across various technologies including Frame Relay, ATM, Ethernet, SONET, and IP-based networks. Its primary goal is to offer priority, dedicated bandwidth, and controlled jitter and latency for certain traffic classes.

Explanation

QoS is needed to meet the specific requirements of applications that demand certain bandwidth, delay, jitter, and loss characteristics. To guarantee these requirements, resources must be reserved at every hop along the end-to-end path, because the network layer bridges per-hop behavior (handled by the data link layer) and end-to-end behavior (handled by the transport layer). The main QoS parameters are bandwidth, delay, jitter, and loss, and different flows (e.g., machine-to-machine, process-to-process, application-to-application) may need different levels of service.

Example

In YouTube video streaming, the client expects a continuous video stream without quality drops or re-buffering. Achieving this experience requires the network to provide sufficient bandwidth and low delay; otherwise the user perceives degraded quality, reflecting a lack of QoS guarantees from the service provider.

Key points

- QoS ensures that selected traffic receives priority, dedicated bandwidth, and controlled jitter and latency.
- Guarantees are achieved by reserving resources per hop along the end-to-end path.
- The four primary QoS parameters are bandwidth, delay, jitter, and loss.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

- Different application classes (e.g., constant-bit-rate voice, real-time variable-bit-rate video conferencing, non-real-time video streaming, best-effort file transfer) require different QoS levels.
- Without a dedicated line or service agreement, typical Internet connections cannot provide strict QoS, leading to performance issues such as buffering in video streams.

Source and licence

This PDF is adapted from **NOC:Computer Networks and Internet Protocol (NPTEL course book, weeks 3, 5, 7)** (https://nptel.ac.in/courses/106105183), accessed 2026-09-26.

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).